CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE

# LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
# RELATÓRIO: PRÁTICA 4

Belo Horizonte, 25 de Setembro de 2025

1.

Programa ex01.0.asm

```
    #include <p18f4550.inc>

    #define VA 0x11     ; Valor de VA
    #define VB 0x03     ; Valor de VB
    #define FA 0x00     ; Endereço de VA
    #define FB 0x01     ; Endereço de VB
    #define FC 0x02     ; Endereço do resultado

    org 0x0000
    bra    start

    org 0x0020
start:
    MOVLW VA        ; Carrega VA
    MOVWF FA, 0
    MOVLW VB        ; Carrega VB
    MOVWF FB, 0
    CLRF  FC, 0     ; Carrega FC (resultado)

    MOVF FA, F      ; Se FA = 0, código encerra
    BZ out
    MOVF FB, F      ; Se FB = 0, código encerra
    BZ out

    bra operate     ; se FA != 0 ^ FB != 0, operamos

operate:
    MOVF FA, W      ; W <- FA (para comparação na intrução seguinte)
    CPFSGT FB       ; FA > FB ?
    bra mul_VA_VB   ; Multiplicar Somando VA e decrementando VB (VB <= VA)
    bra mul_VB_VA   ; Multiplicar Somando VB e decrementando VA (VA < VB)
mul_VA_VB:
    ADDWF FC, F     ; FC <- FC + FA
    DECF FB         ; FB <- FB - 1
    BNZ mul_VA_VB   ; para quando FB ficar zerado
    bra out
mul_VB_VA:
    MOVF FB, W
    ADDWF FC, F     ; FC <- FC + FB
    DECF FA         ; FA <- FA - 1
    BNZ mul_VB_VA   ; para quando FA ficar zerado
out:
    bra out
    end
```

A implementação da operação de multiplicação nesse programa é feita a partir de decrementações e somas sucessivas.

Consideremos então **FC** = **FA** $\times$ **FB**:

Inicializamos os endereços com seus respectivos valores e **FC**, o endereço do resultado, é resetado, em seguida verificamos se **FA** ou **FB** são 0, se sim, pulamos para o fim do programa, sem a necessidade de realizar operações de multiplicação onde um dos termos é 0.

Antes de iniciar o loop de operação, verificamos qual valor é o menor, **VA** ou **VB**, fazemos isso para minimizar a quantidade de iterações, haja vista que estamos decremento o menor valor. Consideremos um exemplo:

Se *VA = 0xFF* e *VB = 0x01*

Caso escolhermos decrementar **VA** e somar **VB** sucessivas vezes, estaríamos utilizando 255 iterações na operação, tornando o programa computacionalmente ineficiente, ao contrário, decrementar **VB** e somar **VA** levaria apenas 1 iteração.

Pelas diretivas do programa, considerando *VA = 0x11* (17) e *VB = 0x03* (3), teremos 3 iterações e o resultado esperado para FC será *0x33* (51)

Ao final do programa, os seguintes valores na memória de dados são mostrados na Figura 1:

| Address | 00 | 01 | 02 | 03 | 04 | 05 |
|---|---|---|---|---|---|---|
| 000 | 11 | 00 | 33 | 00 | 00 | 00 |
| 010 | 00 | 00 | 00 | 00 | 00 | 00 |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 |

*Figura 1:. Endereço FC, destacado em azul, com resultado esperado.*

Pensando em otimizar o programa do exercício 3, pensamos em um novo método para multiplicar números de forma mais otimizada e que aceita operações de valores *signed*:

Programa ex01.1.asm

```
    #include <p18f4550.inc>

    #define FA      0x00
    #define FB      0x01
    #define FC      0x02

    #define COUNT   0x03
    #define SIGN    0x04

    #define VA      0x0A
    #define VB      0x03

    org 0x0000
    bra start

    org 0x0020
start:
    MOVLW   VA        ; Carrega variáveis
    MOVWF   FA, 0
    MOVLW   VB
    MOVWF   FB, 0
    CLRF    FC
    ; Verifica se é multiplicação por 0
    MOVF    FA, W
    BZ      out
    MOVF    FB, W
    BZ      out
```

```
    ; Inicializa variáveis auxiliares
    CLRF    SIGN
    MOVLW   0x08
    MOVWF   COUNT

    MOVF    FA, W
    BNN     next_step   ; Verificar se FA < 0, se sim, FA <- |VA|
    COMF    FA, F
    INCF    FA, F
    BSF     SIGN, 7
next_step:
    MOVF    FB, W
    BNN     end_step    ; Verificar se FB < 0, se sim, FB <- |VB|
    COMF    FB, F
    INCF    FB, F
end_step:
    ANDLW 0x80
    XORWF SIGN, F       ; Sinal de FC será FA[bit7] XOR FB[bit7]
mul_loop:
    BTFSS FB, 0         ; Se B[bit0] = 1, FC += FA
    bra shift
    MOVF FA, W
    ADDWF FC, F
shift:
    BCF STATUS, C
    RLCF FA, F          ; FA * 2
    RRCF FB, F          ; FB / 2
    DECFSZ COUNT, F     ; Condição de parada (COUNT == 0)
    bra mul_loop
    ; Ajuste de sinal do resultado
    BTFSS SIGN, 7
    bra out
    COMF FC, F
    INCF FC, F
out:
    bra out
    end
```

O método utilizado nesse novo programa é análogo a multiplicação clássica ensinada no ensino fundamental, porém com noções na base binária. A ideia é analisar o **LSB** de **FB** (multiplicador), se for 1, **FC** recebe *FC* + *FA*, caso contrário nada é feito, por fim rotacionamos o endereço do multiplicador e repetimos tudo novamente para cada *bit* restante. Logo, existe uma quantidade fixa de 8 iterações de operação, para esse exercício.

Vale ressaltar que é importante limpar o *CARRY* antes da rotação, pois a instrução de soma **ADDWF**, anterior a instrução de rotação **RLCF**, altera o registrador de *STATUS*, podendo comprometer o resultado final, haja vista que, o *CARRY* recebe o **LSB** do endereço rotacionado, comprometendo a informação originais nos endereços.

Para operar números *signed*, avaliamos os sinais de **FA** e **FB**, determinando qual será o sinal final de **FC**, para isso a variável **SIGN**, recebe, antes da operação, o resultado da operação *XOR* entre os *bits* de sinal de **FA** e **FB**, determinando o sinal final de **FC**. Além disso, caso **FA** ou **FB** sejam negativos, trocamos seus valores pelo seu módulo, deixando a operação mais simplificada e corrigindo o sinal ao final do programa.

Pelas diretivas do programa, considerando *VA* = *0xFE* (-2) e *VB* = *0x08* (8), teremos 8 iterações e o resultado esperado para FC será *0xF0* (-16)

Ao final do programa, os seguintes valores na memória de dados são mostrados na Figura 2:

| Address | 00 | 01 | 02 | 03 | 04 | 05 |
|---|---|---|---|---|---|---|
| 000 | 00 | 40 | F0 | 00 | 80 | 00 |
| 010 | 00 | 00 | 00 | 00 | 00 | 00 |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 |

*Figura 2:. Endereço FC, destacado em azul, com resultado esperado*

2. Programa ex02.asm

```
    #include <p18f4550.inc>

    #define V0 0            ; Fibonacci(0)
    #define V1 1            ; Fibonacci(2)
    ; Endereço 0x00 -> Fibonacci(N - 2)
    ; Endereço 0x01 -> Fibonacci(N - 1)
    #define COUNTER 0x02    ; Endereço do contador (0 -> 13)

    org 0x0000
    bra    start
    org 0x020

start:
    MOVLB 0x03          ; Seleção do Banco 3
    MOVLW (0x0D - 2)    ; Inicializando COUNTER COM 13
    MOVWF COUNTER, 0
    LFSR FSR0, 0x300    ; Inicializando FSR0
    MOVLW V0            ; Carregando V0
    MOVWF POSTINC0
    MOVWF 0x00, 0       ; Inicializando F(n - 2)
    MOVLW V1            ; Carregando V1
    MOVWF POSTINC0
    MOVWF 0x01, 0       ; Inicializando F(n - 1)
repeat:
    MOVF 0x00, W        ; W <- F(n - 2)
    ADDWF 0x01, W, 0    ; W <- F(n - 2) + F(n - 1)

    MOVFF 0x001, 0x000  ; F(n - 2) <- F(n - 1)
    MOVWF 0x01, 0       ; F(n - 1) <- W
    MOVWF POSTINC0      ; Atualizando Sequência no banco

    DECF COUNTER        ; Decrementa COUNTER até zerar
    BNZ repeat
out:
    bra out
    end
```

Para esse exercício, denotaremos **F(n)** o n-ésimo elemento da sequência de Fibonacci, correspondente em assembly, aos registradores **POSTINC0**.

O Programa em questão gera os 13 primeiros elementos da sequência de Fibonacci, o vetor que armazena a sequência de número se encontra no Banco 3, do endereço *0x00* até *0x0B*. Além disso, os endereços *0x00*, *0x01* e *0x02* do banco de acesso, são utilizados como auxiliares para geração da sequência, correspondendo, respectivamente, ao valor *N - 2* e *N - 1* da sequência de Fibonacci e ao contador que contará a partir de 11, encerrando o programa quando chegar em 0.

Inicialmente definimos o BSR como 3, pois trabalharemos, em algumas partes, com o Banco 3. Inicializamos o contador com valor (13 - 2), pois os dois primeiros elementos da sequência são conhecidos: *{0, 1},* logo, o contador deve iniciar 2 passos à frente, a fim de gerar 13 elementos ao todo. Em seguida, FSR0, **V0**, **V1**, **F(0)** e **F(1)** são inicializados com os valores já mencionados.

A parte que se repete no programa consiste em armazenar o valor **V0 (F(N - 2))** no acumulador, somar esse valor com **V1 (F(N - 1))**, armazenando o resultado no acumulador. Adiante, o valor de **V0** recebe **V1** e **V1** recebe o resultado da soma, armazenado no acumulador, por fim, esse mesmo valor é passado para **POSTINC0**, no qual é possível armazenar o **F(n)** e incrementar **FSR0** para a próxima iteração, em uma única instrução.

Ao fim do loop, decrementa COUNTER, que repetirá o loop, enquanto o contador for diferente de 0.

Espera-se que a sequência de valores seja:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, em decimal.

*0h, 1h, 1h, 2h, 3h, 5h, 8h, Dh, 15h, 22h, 37h, 59h, 90h*, em hexadecimal.

Ao final, os seguintes valores na memória de dados são mostrados na Figura 3:

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 0A | 0B | 0C | 0D | 0E | 0F | ASCII |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 2D0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 2E0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 2F0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 300 | 00 | 01 | 01 | 02 | 03 | 05 | 08 | 0D | 15 | 22 | 37 | 59 | 90 | 00 | 00 | 00 | ..... ."7Y.... |
| 310 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 320 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 330 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |
| 340 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | 00 | ..... ..... |

*Figura 3:. Os 13 primeiros endereços do banco 3, destacado em azul, compondo a sequência de fibonacci como esperado.*

3. Programa ex03.asm

```
    #include <p18f4550.inc>

    #define VN          0x32    ; Número binário avaliado
    #define FN          0x00    ; Endereço do avaliado
    #define FMASK       0x01    ; Endereço da máscara de bits
    #define FCONTROL    0x02    ; Endereço da variável de controle
    #define FCOUNT      0x03    ; Endereço do contador

    org 0x0000
    bra    start
    org 0x020

start:
    MOVLW VN            ; Carregar avaliado
    MOVWF FN, 0
    CLRF FCONTROL       ; Limpar variável de controle
    MOVLW 0x01
    MOVWF FMASK, 0      ; Inicializar Máscara como 00000001
    MOVLW 0x08          ; Contador inicializado em 8 (varredura de 8 bits)
    MOVWF FCOUNT, 0
repeat:
    MOVF FN, W          ; Filtra Bit do avaliado
    ANDWF FMASK, W
    BZ bit_pair
bit_odd:
    BTG FCONTROL, 0     ; Se bit é ímpar inverte o valor da variável de controle
bit_pair:
    RLNCF FMASK, F, 0   ; Para a próxima iteração a mascará roda
    DECF FCOUNT         ; Contador decrementa
    BNZ repeat          ; Continua até contador zerar
    MOVF FCONTROL, W
    BNZ set_bit         ; Se a variável de controle é 1 (Nº de bits ÍMPAR) Seta LSB
    BCF, 0              ; Se a variável de controle é 0 (Nº de bits PAR) Reseta LSB
    bra out
set_bit:
    BSF, 0
out:
    bra out
    end
```

Denotaremos, para esse programa, **VN** como o valor do número avaliado e **FN** seu endereço. **FMASK**, **FCONTROL** e **FCOUNT** são variáveis auxiliares na execução do programa e serão apresentadas conforme o programa é descrito.

Para avaliar a quantidade de *bits* 1 de um número de 8 *bits*, optamos pela seguinte lógica: primeiro utilizamos uma máscara de *bits*, armazenada em **FMASK**, que filtra 1 *bit* por vez da esquerda para a direita. Para isso a máscara deve iniciar com valor literal *0x01* e a medida que os *bits* são lidos, ela deve ser rotacionada para a direita uma única vez. A operação lógica *AND* entre o número avaliado e a máscara, resultará sempre em número igual ou diferente de 0.

Verifiquemos com exemplos:

Para *bit* 0:

$$\begin{array}{ll} & 01001110 \\ AND & 00000001 \\ \hline & 00000000 \end{array}$$

<- *Máscara para filtrar LSB*

= *0x00* <- *Resultado é sempre 0 se bit avaliado é 0*

Para *bit* 1:

$$\begin{array}{ll} & 00011101 \\ AND & 00010000 \\ \hline & 00010000 \end{array}$$

<- *Máscara para filtrar BIT4*

= *0x10* <- *Resultado é sempre ≠0 se bit avaliado é 1*

Com base nessa informação, podemos identificar, utilizando a instrução condicional **BZ** / **BNZ** o nível lógico dos 8 *bits* do avaliado, separando entre *bit* par(0) ou ímpar(1).

Para contar a quantidade de *bits* ímpares utilizamos a instrução **BTG** no endereço **FCONTROL**, para cada *bit* 1, invertemos o nível lógico do **LSB** de **FCONTROL** tornando *0x00* ou *0x01*. Se **FCONTROL** for inicializado com *0x00*, e ao final das iterações continuar *0x00*, a quantidade de *bits* 1 do número é par, caso seja *0x01*, a quantidade será ímpar. Logo, bastará tomar a decisão de *setar* ou *resetar* o **LSB** do avaliado, com base em **FCONTROL**.

Por fim, um contador de 8 a 0, **FCOUNT**, é utilizado para varrer os 8 bits do avaliado.

Tomando **VN** como *0x32* (00110010), o **LSB** deve ser setado, pois há 3 *bits* (1) tornando **VN** igual a *0x33* (00110011).

Ao final do programa o novo valor de **VN** é mostrado na Figura 4:

| Address | 00 | 01 | 02 | 03 | 04 | 05 | 0 |
|---|---|---|---|---|---|---|---|
| 000 | 33 | 01 | 01 | 00 | 00 | 00 | 0 |
| 010 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |
| 020 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |
| 030 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |
| 040 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |
| 050 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |
| 060 | 00 | 00 | 00 | 00 | 00 | 00 | 0 |

*Figura 4: Endereço FN, destacado em azul, com valor 0x33 para VN inicial de 0x32*